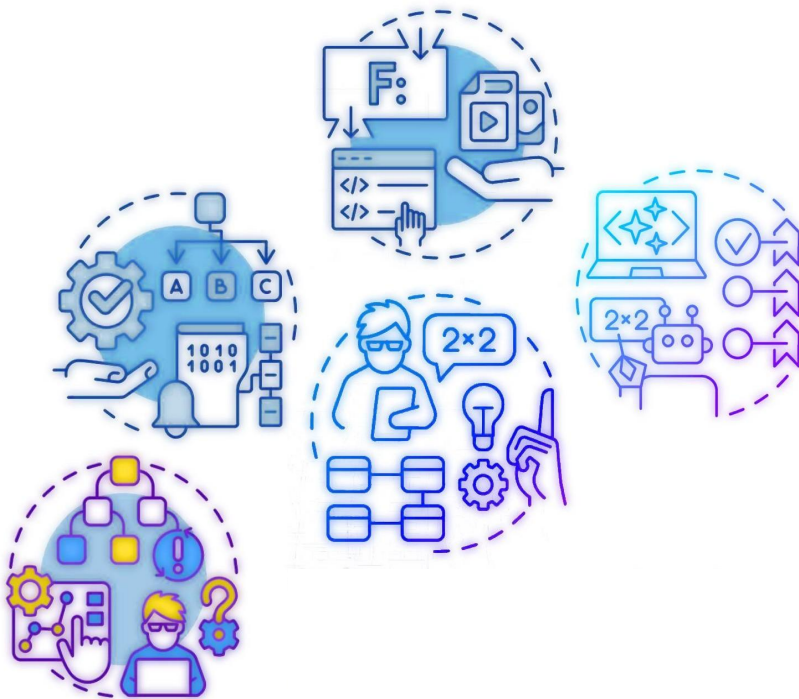




Paradigma Funcional

Esp. Ing. Viviana A. Santucci
AIA Federico Castoldi
Ing. J. Exequiel Benavidez
Ing. Jimena Bourlot

Tecnologías de la Programación

1 – Funciones de Orden Superior:

Las funciones map, for-each, filter, foldl y foldr se consideran funciones de orden superior porque cumplen al menos una de las siguientes condiciones:

- Aceptan una o más funciones como argumentos.
- Devuelven una función como resultado.

1 – Funciones de Orden Superior: map

- `map(función, colección)`: La función `map` toma como primer argumento una función (la función de transformación) que se aplicará a cada elemento de la colección. Por lo tanto, cumple la primera condición de ser una función de orden superior.

`map`: Transformando cada elemento

Se tiene una lista de números y se quiere obtener el doble de cada uno. En la programación imperativa, probablemente se escribiría un bucle. En el paradigma funcional, la función `map` viene al rescate.

1 – Funciones de Orden Superior: map

map toma dos argumentos:

1. Una función de transformación: Esta función se aplica a cada elemento de la colección.
2. Una colección (como una lista, un conjunto, etc.).

El resultado de map es una nueva colección que contiene los resultados de aplicar la función de transformación a cada elemento de la colección original.

1 – Funciones de Orden Superior: map

Ejemplo conceptual:

Si se tiene la lista [1, 2, 3] y queremos duplicar cada elemento, aplicaríamos una función que toma un número y devuelve su doble.

map aplicaría esta función a 1, luego a 2, y finalmente a 3, produciendo una nueva lista: [2, 4, 6].

En esencia, map permite realizar una operación uniforme sobre todos los elementos de una colección sin necesidad de bucles explícitos.

1 – Funciones de Orden Superior: for-each

`for-each` es similar a `map` excepto que `for-each` no crea ni devuelve una lista de los valores resultantes, y `for-each` garantiza realizar las aplicaciones en secuencia sobre los elementos de izquierda a derecha.

El `procedimiento` debería aceptar tantos argumentos como listas haya y no debería mutar los argumentos de la `lista`.

1 – Funciones de Orden Superior: for-each

Propósito: La función `for-each` se utiliza para **iterar sobre los elementos de una colección y ejecutar una función (a menudo con efectos secundarios) para cada elemento**.

Resultado: A diferencia de `map`, `for-each` **generalmente no devuelve un nuevo valor o colección útil directamente**. Su principal objetivo es realizar acciones o efectos secundarios para cada elemento. En muchos lenguajes funcionales, `for-each` puede devolver `void` o una unidad (un valor que indica la ausencia de un resultado significativo).

Efectos Secundarios: La función que se pasa a `for-each` **a menudo se utiliza para causar efectos secundarios**, como imprimir en la consola, modificar estado externo (aunque esto se suele evitar en la programación funcional pura), o realizar operaciones de E/S.

Analogía: `for-each` sería como tener una cesta de manzanas y para cada manzana, olerla y decir en voz alta "esta es una manzana". La cesta de manzanas original no se transforma, y no se crea una nueva cesta. La acción se realiza *para cada* manzana.

1 – Funciones de Orden Superior: for-each

for-each se utiliza con más precaución en la programación funcional pura, ya que su enfoque en los efectos secundarios puede ir en contra de los principios de pureza de funciones y transparencia referencial.

Sin embargo, sigue siendo útil para realizar operaciones de E/S o interacciones con el mundo exterior después de haber realizado las transformaciones necesarias con **map**, **filter**, **fold**, etc.

En muchos lenguajes funcionales, se prefiere utilizar construcciones como la comprensión de listas o las funciones **map** y **filter** para lograr los mismos objetivos que un bucle **for** tradicional con efectos secundarios. **for-each** se reserva a menudo para los casos donde el efecto secundario es el objetivo principal.

1 – Funciones de Orden Superior: map / for-each

Diferencias Clave en Resumen:

Característica	map	for-each (o forEach)
Propósito	Transformar cada elemento de una colección	Ejecutar una acción para cada elemento
Valor de Retorno	Nueva colección con los resultados	Generalmente <code>void</code> o un valor no significativo
Efectos Secundarios	Idealmente sin efectos secundarios en la función de transformación	A menudo se utiliza para causar efectos secundarios en la función aplicada

1 – Funciones de Orden Superior: filter

filter: Seleccionando los elementos que cumplen una condición.

Se tiene una lista de números y solo se quiere conservar los que son pares. Aquí es donde entra en juego la función filter.

filter toma dos argumentos:

- Un predicado (una función que devuelve un valor booleano): Este predicado se evalúa para cada elemento de la colección.
- Una colección.

El resultado de filter es una nueva colección que contiene sólo los elementos de la colección original para los cuales el predicado devuelve True.

1 – Funciones de Orden Superior: filter

Ejemplo conceptual:

Si se tiene la lista [1, 2, 3, 4, 5, 6] y se quiere filtrar los números pares, se aplicaría un predicado que verifica si un número es divisible por 2. filter evaluaría este predicado para cada número y sólo incluiría en la nueva lista aquellos para los que el predicado es verdadero: [2, 4, 6].

filter permite crear subconjuntos de una colección basados en una condición específica, también sin la necesidad de bucles manuales.

1 – Funciones de Orden Superior: foldl (Fold Left)

foldl es una operación para combinar todos los elementos de una colección en un único valor. Funciona iterando sobre la colección de izquierda a derecha, manteniendo un acumulador.

Toma tres argumentos:

1. Una función de combinación: Esta función toma dos argumentos: el acumulador actual y el elemento actual de la colección. Devuelve el nuevo valor del acumulador.
2. Un valor inicial para el acumulador.
3. Una colección.

1 – Funciones de Orden Superior: foldl (Fold Left)

Proceso conceptual:

1. El acumulador se inicializa con el valor inicial.
2. La función de combinación se aplica al acumulador y al primer elemento de la colección. El resultado se convierte en el nuevo valor del acumulador.
3. Este proceso se repite para el segundo elemento, el tercero, y así sucesivamente, hasta que se haya procesado toda la colección.
4. Finalmente, **foldl** devuelve el valor final del acumulador.

1 – Funciones de Orden Superior: foldl (Fold Left)

Ejemplo

`(foldl cons '() '(1 2 3 4)) devuelve '(4 3 2 1).`

- **foldl**: Aplica una función de forma acumulativa a los elementos de una lista de **izquierda a derecha**.
- **cons**: La función que construye un nuevo par donde el primer argumento se convierte en el primer elemento de la lista resultante, y el segundo argumento es el resto de la lista.
- `'()`: El valor inicial del acumulador (una lista vacía).
- `'(1 2 3 4)`: La lista de entrada.

1 – Funciones de Orden Superior: foldl (Fold Left)

Proceso paso a paso:

1. `foldl` comienza con el valor inicial del acumulador, `'()`, y el primer elemento de la lista, `1`. Aplica `cons`: `(cons '() 1) => (1)` El nuevo acumulador es ahora `'(1)`.
2. Luego, toma el acumulador actual, `'(1)`, y el siguiente elemento de la izquierda, `2`. Aplica `cons`: `(cons '(1) 2) => (2 1)` El nuevo acumulador es ahora `'(2 1)`.
3. A continuación, toma el acumulador actual, `'(2 1)`, y el siguiente elemento, `3`. Aplica `cons`: `(cons '(2 1) 3) => (3 2 1)` El nuevo acumulador es ahora `'(3 2 1)`.
4. Finalmente, toma el acumulador actual, `'(3 2 1)`, y el último elemento, `4`. Aplica `cons`: `(cons '(3 2 1) 4) => (4 3 2 1)` El valor final del acumulador, `'(4 3 2 1)`, es el resultado de la expresión `foldl`.

1 – Funciones de Orden Superior: foldr (Fold Right)

foldr es similar a **foldl**, pero la diferencia clave radica en el orden en que se procesan los elementos de la colección: **de derecha a izquierda**.

foldr también toma los mismos tres argumentos que **foldl**:

1. una función de combinación,
2. un valor inicial y
3. una colección.

1 – Funciones de Orden Superior: foldr (Fold Right)

Proceso conceptual:

1. El acumulador se inicializa con el valor inicial.
2. La función de combinación se aplica al último elemento de la colección y al acumulador. El resultado se convierte en el nuevo valor del acumulador.
3. Este proceso se repite para el penúltimo elemento, el antepenúltimo, y así sucesivamente, hasta que se haya procesado el primer elemento.
4. Finalmente, **foldr** devuelve el valor final del acumulador.

1 – Funciones de Orden Superior: foldr (Fold Right)

Ejemplo

(foldr cons '() '(1 2 3 4)) devuelve '(1 2 3 4)

- **foldr**: Aplica una función de forma acumulativa a los elementos de una lista de derecha a izquierda.
- **cons**: Esta es una función fundamental en Lisp y Scheme (y por lo tanto en Racket). **(cons x y)** construye un nuevo par donde **x** es el primer elemento (el "car") y **y** es el resto de la lista (el "cdr"). Si **y** es una lista, **cons** añade **x** al principio de esa lista.
- **'()**: Este es el valor inicial del acumulador, que representa la lista vacía.
- **'(1 2 3 4)**: Esta es la lista de entrada sobre la cual **foldr** va a operar.

1 – Funciones de Orden Superior: foldr (Fold Right)

Proceso paso a paso:

1. `foldr` comienza con el último elemento de la lista, `4`, y el valor inicial del acumulador, `'()`. Aplica `cons`: `(cons 4 '()) => (4)` El nuevo acumulador es ahora `'(4)`.
2. Luego, toma el siguiente elemento de la derecha, `3`, y el acumulador actual, `'(4)`. Aplica `cons`: `(cons 3 '(4)) => (3 4)` El nuevo acumulador es ahora `'(3 4)`.
3. A continuación, toma el elemento `2` y el acumulador actual, `'(3 4)`. Aplica `cons`: `(cons 2 '(3 4)) => (2 3 4)` El nuevo acumulador es ahora `'(2 3 4)`.
4. Finalmente, toma el primer elemento, `1`, y el acumulador actual, `'(2 3 4)`. Aplica `cons`: `(cons 1 '(2 3 4)) => (1 2 3 4)` El valor final del acumulador, `'(1 2 3 4)`, es el resultado de la expresión `foldr`.

En esencia, este ejemplo muestra cómo `foldr` con `cons` y una lista vacía como valor inicial reconstruye la lista original. Empieza por el final y va añadiendo cada elemento al principio de la lista que se va construyendo..

1 – Funciones de Orden Superior: foldl vs. foldr

¿Cuándo usar **foldl** vs. **foldr**?

En muchas operaciones asociativas y conmutativas (como la suma o la multiplicación), **foldl** y **foldr** producirán el mismo resultado. Sin embargo, el orden de procesamiento puede ser importante en los siguientes casos:

- **Operaciones no conmutativas:** Por ejemplo, la resta o la concatenación de listas. El orden en que se aplican los elementos al acumulador afectará el resultado.
- **Eficiencia:** En algunas estructuras de datos (como las listas enlazadas), **foldr** puede ser más eficiente debido a la forma en que se implementan las operaciones.
- **Evaluación perezosa:** En lenguajes con evaluación perezosa, **foldr** puede trabajar con listas infinitas donde **foldl** podría no terminar.

1 – Funciones de Orden Superior

La característica clave que convierte a map, for-each, filter y fold (tanto foldl como foldr) en funciones de orden superior es su capacidad para operar sobre otras funciones, ya sea aceptándolas como argumentos para definir su comportamiento (la transformación en map, la condición en filter, la acumulación en fold) o, en algunos casos (aunque no es tan común para estas funciones específicas en su forma más básica), devolviendo una nueva función como resultado de su aplicación parcial o currificación.

La capacidad de trabajar con funciones como si fueran datos de primera clase es uno de los pilares fundamentales del paradigma de la programación funcional, y las funciones de orden superior son una manifestación directa de este principio. Permiten una abstracción más poderosa, la creación de código más flexible y reutilizable, y facilitan la composición de operaciones complejas a partir de partes más simples.

2 - Creación de abstracciones propias.

En la programación funcional, se busca construir software componiendo funciones puras. Las funciones de orden superior elevan esta capacidad de composición a un nivel superior, permitiendo abstraer patrones de comportamiento y crear bloques de construcción reutilizables y altamente configurables.

¿Qué significa crear abstracciones propias con funciones de orden superior?

Significa que, en lugar de limitarnos a las funciones de orden superior predefinidas como `map`, `for-each`, `filter` y `fold`, tenemos la capacidad de definir *nuestras propias funciones* que operan sobre otras funciones. Esto permite encapsular lógicas complejas y repetitivas en funciones que pueden ser parametrizadas con diferentes comportamientos (a través de las funciones que reciben como argumentos).

2 - Creación de abstracciones propias.

Beneficios clave de crear abstracciones con funciones de orden superior:

- **Reutilización de código:** Al abstraer patrones comunes, evitamos la repetición de código. Una función de orden superior bien diseñada puede aplicarse a una variedad de situaciones simplemente pasándole diferentes funciones como argumentos.
- **Mayor expresividad:** El código se vuelve más conciso y declarativo. En lugar de detallar los pasos de un proceso, describimos la transformación o la operación de alto nivel que queremos realizar.
- **Mejor legibilidad:** Al descomponer la lógica en funciones más pequeñas y enfocadas, y al utilizar funciones de orden superior para orquestarlas, el código se vuelve más fácil de entender y mantener.
- **Composición poderosa:** Las funciones de orden superior facilitan la composición de comportamientos complejos a partir de funciones más simples. Podemos encadenar o combinar funciones de orden superior para lograr resultados sofisticados.
- **Flexibilidad y extensibilidad:** Al parametrizar el comportamiento con funciones, nuestras abstracciones se vuelven más flexibles y adaptables a diferentes necesidades sin necesidad de modificar el código base.

2 - Creación de abstracciones propias.

Patrones comunes para crear abstracciones con funciones de orden superior:

1. **Generalización de operaciones sobre colecciones:** Ya hemos visto `map`, `filter` y `fold`. Se pueden crear funciones que generalicen otras formas de procesar colecciones. Por ejemplo, una función que tome una colección y una función de dos argumentos para combinar elementos adyacentes.

```
(define (aplicar-varias-veces funcion valor-inicial n)
  (if (zero? n)
      valor-inicial
      (aplicar-varias-veces funcion (funcion valor-inicial) (sub1 n))))
```

```
(define incrementar (lambda (x) (+ x 1)))
(aplicar-varias-veces incrementar 5 3) ; => 8
```

```
(define multiplicar-por-dos (lambda (x) (* x 2)))
(aplicar-varias-veces multiplicar-por-dos 1 4) ; => 16
```


2 - Creación de abstracciones propias.

2. Encapsulación de lógica de control: se pueden crear funciones de orden superior que controlen el flujo de ejecución basándose en otras funciones (predicados). Por ejemplo, una función unless que ejecute una acción sólo si un predicado es falso.

```
(define (filtrar-segun-propiedad lista obtener-propiedad valor-esperado)
  (filter (lambda (objeto) (equal? (obtener-propiedad objeto) valor-esperado)) lista))

(define personas
  (list (hash "nombre" "Ana" "edad" 30)
        (hash "nombre" "Juan" "edad" 25)
        (hash "nombre" "Pedro" "edad" 30)))

(define obtener-edad (lambda (persona) (hash-ref persona "edad")))
(filtrar-segun-propiedad personas obtener-edad 30)
; => '(#hash(("edad" . 30) ("nombre" . "Ana")) #hash(("edad" . 30) ("nombre" . "Pedro")))
```

2 - Creación de abstracciones propias.

2. Encapsulación de lógica de control: se pueden crear funciones de orden superior que controlen el flujo de ejecución basándose en otras funciones (predicados). Por ejemplo, una función unless que ejecute una acción sólo si un predicado es falso.

```
(define (filtrar-segun-propiedad lista obtener-propiedad valor-esperado)
  (filter (lambda (objeto) (equal? (obtener-propiedad objeto) valor-esperado)) lista))

(define personas
  (list (hash "nombre" "Ana" "edad" 30)
        (hash "nombre" "Juan" "edad" 25)
        (hash "nombre" "Pedro" "edad" 30)))

(define obtener-edad (lambda (persona) (hash-ref persona "edad")))
(filtrar-segun-propiedad personas obtener-edad 30)
; => '(#hash(("edad" . 30) ("nombre" . "Ana")) #hash(("edad" . 30) ("nombre" . "Pedro")))
```

2 - Creación de abstracciones propias.

3. Creación de "estrategias" parametrizables: se pueden definir algoritmos donde ciertos pasos o decisiones se delegan a funciones pasadas como argumentos. Esto permite cambiar el comportamiento del algoritmo sin modificar su estructura general.

```
(define (ejecutar-con-log funcion argumento)
  (displayln (format "Ejecutando: ~a con argumento: ~a" (object-name funcion)
                    argumento))
  (let ((resultado (funcion argumento)))
    (displayln (format "Resultado: ~a" resultado))
    resultado))

(define sumar-uno (lambda (x) (+ x 1)))
(ejecutar-con-log sumar-uno 10)
; Imprimirá:
; Ejecutando: #[procedure ...] con argumento: 10
; Resultado: 11
; y devolverá 11
```

2 - Creación de abstracciones propias.

4. Implementación de patrones de diseño funcionales: Muchos patrones de diseño tradicionales pueden ser implementados de manera más elegante y concisa utilizando funciones de orden superior (por ejemplo, el observador).

Conclusión:

La creación de abstracciones propias con funciones de orden superior es una técnica esencial en la programación funcional. Permite ir más allá de las herramientas básicas y construir soluciones elegantes, reutilizables y altamente adaptables a problemas específicos. Al pensar en términos de funciones que operan sobre otras funciones, se puede elevar el nivel de abstracción y escribir código más poderoso y mantenible.

3 - Estructuras de Datos: Vectores inmutables

Un vector inmutable es aquel cuyo contenido no puede ser modificado una vez que ha sido creado. Cualquier operación que parezca "modificar" un vector inmutable en realidad crea y devuelve un nuevo vector con los cambios deseados, dejando el vector original intacto.

Los vectores inmutables se crean utilizando la sintaxis `#(...)`.

- (define mi-vector-inmutable `#(1 2 3 4 5)`)

Se accede a los elementos de un vector inmutable utilizando la función **vector-ref**, que toma el vector y el índice (base 0) como argumentos

- (vector-ref mi-vector-inmutable 2) ; Devuelve 3 (el elemento en el índice 2)

3 - Estructuras de Datos: Vectores inmutables

"Modificación": No existe una función directa para modificar un vector inmutable *in-place*. Si se necesita un vector con un elemento cambiado, se debe crear un nuevo vector. Por ejemplo, para "cambiar" el elemento en el índice 2 a 10, se podría hacer algo como:

```
(define nuevo-vector-inmutable
  (build-vector (vector-length mi-vector-inmutable)
    (lambda (i)
      (if (= i 2)
          10
          (vector-ref mi-vector-inmutable i)))))

nuevo-vector-inmutable ; Devuelve #(1 2 10 4 5)
mi-vector-inmutable    ; Sigue siendo #(1 2 3 4 5)
```

build-vector crea un nuevo vector basado en una función que define cada elemento. El vector original **mi-vector-inmutable** no se modificó.

3 - Estructuras de Datos: Vectores inmutables

Beneficios:

- **Seguridad:** La inmutabilidad facilita el razonamiento sobre el código, ya que puedes estar seguro de que el valor de un vector inmutable no cambiará inesperadamente.
- **Compartición segura:** Los vectores inmutables pueden ser compartidos de forma segura entre diferentes partes del programa sin temor a efectos secundarios no deseados.
- **Programación funcional:** La inmutabilidad es un concepto fundamental en la programación funcional, ya que promueve la transparencia referencial y la evitación de estados mutables.

3 - Estructuras de Datos: Vectores mutables

Un vector mutable es aquel cuyo contenido **puede ser modificado directamente** después de su creación. Las operaciones de modificación alteran el vector original *in-place*.

Creación: Los vectores mutables se crean utilizando la función **vector**.

- (define mi-vector-mutable (vector 1 2 3 4 5))

Acceso: Al igual que con los vectores inmutables, se accede a los elementos con **vector-ref** en tiempo constante.

- (vector-ref mi-vector-mutable 3) ; Devuelve 4

3 - Estructuras de Datos: Vectores mutables

Modificación: Los vectores mutables se modifican utilizando la función `vector-set!`, que toma el vector, el índice y el nuevo valor como argumentos.

La convención de Racket es usar un signo de exclamación (!) al final de los nombres de las funciones que tienen efectos secundarios (como la mutación). Esta operación también tiene un costo de tiempo constante

- `(vector-set! mi-vector-mutable 2 10)`
- `mi-vector-mutable` ; Ahora contiene `#(1 2 10 4 5)` - el vector original ha sido modificado

3 - Estructuras de Datos: Vectores mutables

Modificación: Los vectores mutables se modifican utilizando la función `vector-set!`, que toma el vector, el índice y el nuevo valor como argumentos.

La convención de Racket es usar un signo de exclamación (!) al final de los nombres de las funciones que tienen efectos secundarios (como la mutación). Esta operación también tiene un costo de tiempo constante

- `(vector-set! mi-vector-mutable 2 10)`
- `mi-vector-mutable` ; Ahora contiene `#(1 2 10 4 5)` - el vector original ha sido modificado

3 - Estructuras de Datos: Vectores mutables

Beneficios:

- **Eficiencia:** La mutación *in-place* puede ser más eficiente en términos de memoria y rendimiento para ciertas operaciones, ya que no requiere la creación de nuevas estructuras de datos.
- **Programación imperativa:** Los vectores mutables son más compatibles con el estilo de programación imperativa, donde la modificación del estado es una práctica común.

3 - Estructuras de Datos: Vectores mutables - inmutables

Cuándo usar cuál:

- **Favorece la inmutabilidad:** En general, en el espíritu de la programación funcional, se recomienda **favorecer el uso de vectores inmutables** siempre que sea posible. Esto conduce a un código más predecible, seguro y fácil de razonar.
- **Usa la mutabilidad con cuidado:** Los vectores mutables pueden ser útiles en situaciones donde la eficiencia es crítica y la mutación se realiza de manera controlada y bien entendida, como en algoritmos que requieren actualizaciones *in-place* para optimizar el rendimiento. Sin embargo, es importante ser consciente de los posibles efectos secundarios y las dificultades que la mutabilidad puede introducir en el razonamiento sobre el programa.

En resumen, Racket ofrece ambas formas de trabajar con arreglos de tamaño fijo. La elección entre vectores inmutables y mutables depende de las necesidades específicas de tu programa y de tu preferencia por el estilo de programación funcional o imperativo. Sin embargo, **la filosofía funcional de Racket inclina la balanza hacia el uso de estructuras de datos inmutables siempre que sea práctico.**

4 - Entrada/Salida (E/S) de datos

Desde una perspectiva funcional, la E/S presenta un desafío interesante debido a su naturaleza inherentemente impura (causan efectos secundarios en el "mundo exterior"). Sin embargo, los lenguajes funcionales, incluyendo aquellos como Racket donde estas funciones son comunes, buscan manejar la impureza de manera controlada y separada del núcleo puramente funcional del programa.

4 - Entrada/Salida (E/S) de datos: read-char

read-char: permite leer un carácter desde el puerto indicado. Por defecto, la consola:

- **Impureza:** Esta función es inherentemente impura. Su resultado (el carácter leído) depende de un factor externo al programa (la entrada del usuario o el contenido del puerto). Llamar a `read-char` en diferentes momentos o con diferentes entradas producirá resultados potencialmente diferentes.
- **Perspectiva Funcional:** En un paradigma puramente funcional, una función con la misma entrada siempre debería producir la misma salida. `read-char` viola este principio. Para manejar esto funcionalmente, a menudo se emplean técnicas como la mónada de E/S (IO monad), que encapsula las operaciones impuras y las secuencia de manera controlada, aunque el texto no menciona este nivel de abstracción. Aquí, se presenta como una función que interactúa directamente con el exterior.

4 - Entrada/Salida (E/S) de datos: read-line

read-line: lee toda una línea de caracteres desde el puerto indicado y devuelve un string:

- **Impureza:** Similar a `read-char`, `read-line` también es impura. Su resultado depende de la entrada externa.
- **Perspectiva Funcional:** Al igual que con la lectura de caracteres, la lectura de líneas introduce impureza. En un contexto funcional más avanzado, esto también se manejaría dentro de una estructura que rastrea y controla los efectos secundarios.

```
(read-line [in mode]) → (or/c string? eof-object?)           procedure
  in : input-port? = (current-input-port)
  mode : (or/c 'linefeed 'return 'return-linefeed 'any 'any-one)
         = 'linefeed
```

4 - Entrada/Salida (E/S) de datos: write-char

write-char: escribe un carácter al puerto indicado:

- **Impureza:** Esta función es impura porque causa un efecto secundario: la modificación del estado del "mundo exterior" al escribir un carácter en un puerto (típicamente la consola o un archivo). No devuelve un valor significativo en términos de cálculo puro; su valor reside en el efecto que produce.
- **Perspectiva Funcional:** Las funciones puras no deberían tener efectos secundarios. `write-char` se centra completamente en su efecto secundario. En un manejo funcional más estricto, las operaciones de escritura también se encapsularían dentro de la mónada de E/S para secuenciar y controlar estos efectos.

```
(write-char char [out]) → void?
```

procedure

```
char : char?
```

```
out : output-port? = (current-output-port)
```


4 - Entrada/Salida (E/S) de datos: write

write: escribe la expresión en el puerto indicado en formato de máquina. Ej: los strings con comillas dobles y los caracteres precedidos con #\:

- **Impureza:** Al igual que `write-char`, `write` causa un efecto secundario al escribir una representación de la expresión en un puerto.
- **Perspectiva Funcional:** Aunque la función toma una expresión como entrada, su propósito principal es el efecto secundario de la salida. La "forma de máquina" sugiere una representación interna o más literal de los datos.

```
(write datum [out]) → void? procedure  
  datum : any/c  
  out : output-port? = (current-output-port)
```

4 - Entrada/Salida (E/S) de datos: display

display: muestra el resultado de la expresión:

- **Impureza:** `display` también es una función impura, centrada en el efecto secundario de mostrar el resultado de una expresión en un puerto (generalmente la consola) de una manera más legible para el usuario que `write`.
- **Perspectiva Funcional:** Su valor reside en su efecto visible en el exterior, no en un valor de retorno computable de manera pura.

```
(display datum [out]) → void?           procedure  
  datum : any/c  
  out : output-port? = (current-output-port)
```

4 - Entrada/Salida (E/S) de datos: implicaciones

1. **Separación de lo Puro y lo Impuro:** En la programación funcional, se busca idealmente separar la lógica de cálculo pura (funciones sin efectos secundarios) de las operaciones impuras como la E/S. Esto hace que el núcleo del programa sea más fácil de razonar, testear y componer.
2. **Manejo de Efectos Secundarios:** Los lenguajes funcionales emplean diversas estrategias para manejar los efectos secundarios de manera controlada. Si bien este texto no profundiza en ellas, en contextos funcionales más avanzados se verían técnicas como:
 - **Mónadas de E/S:** Para encapsular y secuenciar operaciones impuras.
 - **Streams y Datos Observables:** Para modelar flujos de entrada y salida como secuencias de datos.

4 - Entrada/Salida (E/S) de datos: implicaciones

3. **Funciones como "Acciones":** Las funciones de E/S pueden verse como "acciones" que interactúan con el mundo exterior. Su composición se convierte en la secuencia de estas acciones.
4. **Importancia del Estado:** La E/S a menudo implica la interacción con un estado mutable externo (el sistema de archivos, la consola, la red). El paradigma funcional busca minimizar el estado mutable dentro del programa, por lo que la interacción con el estado externo se gestiona cuidadosamente.

4 - Entrada/Salida (E/S) de datos: conclusiones

Las funciones descritas son herramientas pragmáticas para permitir que un programa funcional interactúe con el mundo exterior. Reconocen la necesidad de la E/S, que es inherentemente impura.

En un contexto de programación funcional más estricto, estas operaciones se gestionan de manera más abstracta para preservar las ventajas de la pureza funcional en el núcleo del programa.

Sin embargo, en muchos lenguajes funcionales prácticos, estas funciones se proporcionan directamente para facilitar la interacción con el entorno.

5 - Puertos de archivos

En Racket, un **puerto** es una abstracción que representa una fuente o un destino para datos secuenciales.

La interacción con el sistema de archivos es otra área donde la impureza es inherente, ya que involucra efectos secundarios en el estado del sistema de archivos (creación, modificación) y la lectura de datos que no están determinados puramente por las entradas al programa en ese momento.

5 - Puertos de archivos: open-input-file

open-input-file: abre un archivo y devuelve un puerto de lectura:

- **Impureza:** Esta operación es impura. El resultado (el puerto de lectura) depende del estado del sistema de archivos (si el archivo existe, permisos, etc.), que es externo al programa en ese instante. La misma llamada con el mismo nombre de archivo podría tener éxito o fallar en diferentes momentos debido a cambios en el sistema de archivos.
- **Perspectiva Funcional:** Idealmente, una función con la misma entrada debería producir la misma salida. `open-input-file` viola esto. En un manejo funcional más avanzado, la apertura de archivos se podría encapsular dentro de una mónada de E/S para rastrear y controlar estos efectos. El puerto de lectura en sí mismo representa un estado mutable potencial (la posición de lectura dentro del archivo).

5 - Puertos de archivos: open-input-file

```
(open-input-file path                                     procedure
      [#:mode mode-flag
       #:for-module? for-module?]) → input-port?

path : path-string?
mode-flag : (or/c 'binary 'text) = 'binary
for-module? : any/c = #f
```

Ejemplo

```
> (with-output-to-file some-file
   (lambda () (printf "hello world")))
> (define in (open-input-file some-file))
> (read-string 11 in)
"hello world"
> (close-input-port in)
```


5 - Puertos de archivos: open-output-file

open-output-file: abre un archivo y devuelve un puerto de escritura:

- **Impureza:** Esta operación es aún más claramente impura. Causa un efecto secundario en el sistema de archivos (creación o truncamiento del archivo) y devuelve un puerto que se utilizará para futuras operaciones con efectos secundarios (escritura). El éxito o el fracaso dependen del estado externo (permisos, espacio en disco, existencia del directorio).
- **Perspectiva Funcional:** Esta función está completamente enfocada en su efecto secundario. En un paradigma funcional puro, las operaciones de escritura se gestionarían dentro de la mónada de E/S para secuenciar y controlar estos efectos, asegurando un manejo adecuado de los recursos y posibles errores. El puerto de escritura también representa un estado mutable (el estado del archivo al que se está escribiendo).

5 - Puertos de archivos: open-output-file

```
(open-output-file path                                     procedure
    [#:mode mode-flag
     #:exists exists-flag
     #:permissions permissions]
    #:replace-permissions? replace-permissions?)

→ output-port?
path : path-string?
mode-flag : (or/c 'binary 'text) = 'binary
exists-flag : (or/c 'error 'append 'update 'can-update
                  'replace 'truncate
                  'must-truncate 'truncate/replace)
              = 'error
permissions : (integer-in 0 65535) = #o666
replace-permissions? : #f
```

5 - Puertos de archivos: open-output-file

Ejemplos:

```
> (define out (open-output-file some-file))  
> (write "hello world" out)  
> (close-output-port out)
```

5 - Puertos de archivos: close-input-port / close-output-port

close-input-port / close-output-port: cierran los puertos:

- **Impureza:** Estas operaciones también tienen que ver con efectos secundarios, aunque de una manera diferente. Cierran una conexión a un recurso externo (el archivo), lo que podría liberar recursos del sistema operativo. Aunque no producen un valor directamente relevante para el cálculo puro, su efecto en el entorno del programa es importante.
- **Perspectiva Funcional:** Cerrar puertos es esencial para la gestión de recursos. En un contexto funcional, esto podría ser parte de un mecanismo más amplio de gestión de recursos que garantice que los recursos se liberen incluso en caso de errores, similar a la idea de `finally` en el manejo de excepciones en lenguajes imperativos, pero quizás expresado a través de abstracciones funcionales como mónadas de recursos.

5 - Puertos de archivos: close-input-port / close-output-port

```
(close-input-port in) → void?           procedure  
  in : input-port?
```

```
(close-output-port out) → void?         procedure  
  out : output-port?
```

5 - Puertos de string

Los **puertos de string** extienden la idea de puertos de archivos, permitiéndonos tratar una **cadena de caracteres** como si fuera un archivo del cual se puede:

- leer secuencialmente (puertos de entrada de string)
- escribir secuencialmente (puertos de salida de string)

5 - Puertos de string: open-input-string

Un puerto de entrada de string permite leer caracteres y "objetos" (en el sentido de la función `read`) desde una cadena como si estuviéramos leyendo desde un archivo.

- **Creación:** Se crean utilizando la función (`open-input-string cadena`)

donde `cadena` es la cadena de la cual se quiere leer. Esta función devuelve un **puerto de entrada**.

- **Lectura:** Una vez que se tiene un puerto de entrada de string, se pueden utilizar las mismas funciones de lectura que se usan para los puertos de archivo:
 - `read-char`: Lee el siguiente carácter de la cadena.
 - `read`: Lee el siguiente "objeto Scheme" (símbolo, número, cadena, lista, etc.) delimitado por separadores (espacios, tabulaciones, saltos de línea).
 - `read-line`: Lee la siguiente línea completa de la cadena (hasta un salto de línea).
 - `eof-object?`: Permite verificar si hemos llegado al final de la cadena.

5 - Puertos de string: open-input-string

- **No Necesitan Cierre Explícito (Generalmente):** A diferencia de los puertos de archivo que deben cerrarse explícitamente con `close-input-port`, los puertos de entrada de string generalmente se gestionan automáticamente por el recolector de basura de Racket una vez que dejan de ser referenciados. Sin embargo, cerrarlos explícitamente con `close-input-port` es una buena práctica si se desea liberar cualquier recurso asociado de inmediato.

5 - Puertos de string: open-output-string

Un puerto de salida de string permite escribir caracteres y "objetos" en una cadena que se construye dinámicamente en memoria.

- **Creación:** Se crean utilizando la función (open-output-string)

Esta función devuelve un **puerto de salida**.

- **Escritura:** Se puede utilizar las funciones de escritura estándar:
 - `write-char`: Escribe un carácter en el puerto.
 - `write`: Escribe una expresión en formato de máquina.
 - `display`: Muestra el resultado de una expresión.

5 - Puertos de string: open-output-string

- **Obtención de la Cadena Resultante:** Una vez que hemos escrito los datos deseados en el puerto de salida de string, necesitamos obtener la cadena resultante. Esto se hace utilizando la función `get-output-string`:

`(get-output-string puerto-salida)`

Esta función devuelve la cadena que se ha construido en el puerto de salida.

- **No Necesitan Cierre Explícito (Generalmente):** Al igual que con los puertos de entrada de string, el recolector de basura generalmente gestiona los puertos de salida de string. Sin embargo, cerrarlos con `close-output-port` es una buena práctica.

5 - Puertos de string: beneficios

Abstracción de E/S: Los puertos de string permiten tratar las cadenas como flujos de datos, lo que facilita el uso de las mismas funciones de E/S que se utilizan para archivos y otros tipos de puertos. Esto promueve la abstracción y la reutilización de código.

Manipulación de Cadenas Complejas: Son útiles para construir cadenas complejas dinámicamente, especialmente cuando se involucran múltiples pasos de formato o concatenación. En lugar de concatenaciones repetidas (que pueden ser menos eficientes para cadenas grandes), se puede escribir secuencialmente en un puerto de salida de string y obtener el resultado final de una sola vez.

5 - Puertos de string: beneficios

Testing: Los puertos de string son valiosos para realizar pruebas unitarias de funciones que realizan operaciones de E/S. Se puede simular la entrada proporcionando una cadena a un puerto de entrada de string y verificar la salida capturándola desde un puerto de salida de string, sin necesidad de interactuar con el sistema de archivos real o la consola. Esto facilita la creación de pruebas deterministas y aisladas.

Procesamiento de Texto: Facilitan el procesamiento de texto al permitir leer y analizar cadenas secuencialmente, utilizando las mismas herramientas que se usarían para archivos de texto.

5 - Puertos de string: beneficios

En el paradigma funcional, donde la inmutabilidad y la separación de efectos secundarios son importantes, los puertos de string ofrecen una forma controlada de manejar la entrada y salida de datos relacionados con cadenas sin depender de la mutación explícita de las cadenas originales.

La construcción de cadenas se realiza de manera secuencial a través del puerto de salida, y la lectura se realiza de manera secuencial desde el puerto de entrada, manteniendo la naturaleza de flujo de datos.